

```
>arrayArg=00000000000003
Entered [5 ] with st->val=0 , st-
>arrayArg=00000000000000
```

'trace2_mod.gdb'

This script is a modified version of *trace2.gdb*, which shows how one can modify the function arguments. Let us suppose that, arbitrarily, the developer wants *func* to be called with only odd values of *st->val*. The random-number logic may not always satisfy this condition, so this script will capture the function arguments, and change even numbers in *st->val* into odd numbers, by subtracting one from the value, and signal this by emitting a line having "...(*HACK*): ". Please note: the script only changes the numeric value *st->val* and leaves *st->arrayArg* unchanged.

```
set environment LD_PRELOAD ./libauxSym.so
file ./traceMe_rel
br main
run 100
bt
ptype auxST
ptype func
br func
c
while 1
set variable $_one=*((int *)($ebp+0x8))

printf "\n GDB:TRACER: %d, %s", ((auxST *) ($_one))->auxVal,
((auxST *) ($_one))->auxArray
set variable $_integer=((auxST *) ($_one))->auxVal
if $_integer > 0 && $_integer %2 == 0
printf "...(HACK): Not allowing even numbers, old = %d, new
=%d", $_integer, $_integer -1
set ((auxST *) ($_one))->auxVal = $_integer -1
end

c
end
```

Run *trace2_mod.gdb* on *traceMe_rel* with *gdb -x trace2_mod.gdb -batch > out_trace2_mod.txt*. Now, extract the script output with 'grep' on "GDB" in *out_trace2_mod.txt* (#1), and output of *traceMe_rel* with a 'grep' on "Enter" (#4).

We find that *traceMe_rel* called *func* with even numbers in *st->val* twice (100 and 12), as seen in the two lines with "*HACK*" (#2 and #3). These lines show the original value (*old* = 100) passed to *func*, and the changed value (*new* = 99). To see that the state has really been changed, look at lines #5 and #6, comparing *st->val* = 99, but the string version is left as *st->arrayArg* = 100 since *trace2_mod.gdb* does not change *arrayArg*.

```
[raman@Chalotra Part2]$ grep GDB out_trace2_mod.txt #1
GDB:TRACER: 100, 00000000000100...(HACK):
Not allowing even numbers, old = 100, new =9
9 #2
GDB:TRACER: 49, 00000000000049
GDB:TRACER: 12, 00000000000012...(HACK):
Not allowing even numbers, old = 12, new =9
1 #3
GDB:TRACER: 3, 00000000000003
GDB:TRACER: 0, 00000000000000
[raman@Chalotra Part2]$ grep Enter out_trace2_mod.txt #4
Entered [1 ] with st->val=99 , st-
>arrayArg=00000000000100 #5
Entered [2 ] with st->val=49 , st-
>arrayArg=00000000000049
Entered [3 ] with st->val=11 , st-
>arrayArg=00000000000012 #6
Entered [4 ] with st->val=3 , st-
>arrayArg=00000000000003
Entered [5 ] with st->val=0 , st-
>arrayArg=00000000000000
```

This approach is powerful, because GDB provides a huge set of commands to check the state of the program, and hence developers can make a rich set of tests before taking any action. This script just demonstrates a small modification (converting even number to odd) to the inferior; however, great things can be done using this technique. For example, instead of changing the even number to odd, the developer can provide a second version of *func* (let's call it *func_even*) packaged in *libauxSym.so*. On seeing an even number in *st->val*, the script can call *func_even* instead of *func*.

Summing up

In this article, we have discussed how a debugger can be used to perform nice things other than just debugging! I hope you enjoyed and learned something new from this article. The approach suggested here may not be used safely on production applications, but can be used for research and to learn about the various aspects of the system and your application.

Wish you all a happy new year! 

References

'GDB: Logging Function Parameters—Part 1' published in *LFY*, November 2011 edition.

By: Raman Deep

Raman works as a senior software engineer in a company that deals in software security. He has over 6 years of software development experience in C and Linux. He can be reached at rd.golinux@gmail.com.